Smart Refrigerator Monitor — Technical Explainer & Viva Prep

Oct 24, 2026 · @KEVIN JOSEPH

Project Overview

The Smart Refrigerator Health Monitoring & Energy Optimization System is an ESP32-based edge device that watches a refrigerator's temperature, humidity and door behaviour, turns that into a single 0-100 health score, and serves a live dashboard over its own Wi-Fi hotspot — no cloud, no router, no app install.

The problem it solves. A stock refrigerator thermostat shows one number and nothing else: no record of how often the door opens, how long the compartment takes to recover after a door-open event, or whether the compressor is slowly struggling more than it used to. This project adds that missing layer of visibility using low-cost sensors and on-device logic, without touching the compressor or mains wiring.

How it works, in one line. Sensors feed the ESP32 → the ESP32 scores and logs what it sees → a phone or laptop connects directly to the ESP32's own Wi-Fi network (SoftAP) and opens a local webpage to view it live.

Why no cloud or internet. The device runs as its own Wi-Fi access point (SoftAP: `Smart_Refrigerator` / password `refrigerator`), so it works anywhere — a lab bench, a viva room, a home kitchen — with zero dependency on a router, an internet connection, or any external server. This also means no NTP time sync is possible, which shapes how day-tracking and history are implemented (see *Persistence & Day Tracking*).

Explicitly out of scope: food-freshness prediction, cloud dashboards, direct compressor/mains control, and measured (vs. estimated) power consumption — these were deliberately dropped to keep the build achievable with team's hardware experience and the ₹2,500–3,000 budget.

Hardware Components & Their Functions

Component	Pin / Connection	What it does
ESP32-WROOM-32	—	The controller. Runs the firmware, hosts the Wi-Fi access point, and serves the web dashboard.
DS18B20 (waterproof probe)	GPIO 4, 4.7kΩ pull-up to 3V3	Measures refrigerator compartment temperature. The only sensor placed inside the cold chamber, because it's the only waterproof one.
DHT22 / AM2302	GPIO 16	Measures humidity and a secondary ambient temperature reading.
MC-38 magnetic reed switch	GPIO 17, `INPUT_PULLUP`	Detects door open/closed. One half mounts on the door, the other on the frame; the magnetic field breaks the circuit when the door opens.
0.96″ SSD1306 OLED	SDA = GPIO 21, SCL = GPIO 22 (I^2C)	Shows live readings on the device itself — temperature, humidity, door state, health score, estimated kWh/day, day number — so status is visible without a phone.
Active buzzer module	GPIO 25	Sounds when the door has been open past the warning threshold (30s), as a physical nudge to close it.
Breadboard + jumper wires	Common GND	Prototyping connections between all modules.

Why the DS18B20 specifically sits inside the fridge: it's rated waterproof/condensation-safe. Every other module (ESP32, DHT22, buzzer, OLED) stays outside the cold chamber, mounted just outside the door or in the enclosure housing, both to avoid condensation damage and because they don't need to be inside to do their job — door state and ambient conditions can be sensed from just outside the seal.

Safety note: the system only *monitors*. It never switches the compressor or touches mains power, so there's no electrical-safety risk beyond standard low-voltage wiring practice.

Software Architecture

The firmware runs one continuous `loop()` with no blocking delays — every piece of work is gated by its own timer check (`millis() - lastX >= interval`) so the web server stays responsive while sensors are read in the background. Each pass through the loop does, in order:

1. **Acquire** — every 2 seconds, read the DS18B20 and DHT22 without blocking (the DS18B20 conversion is kicked off, then read back on the *next* cycle rather than waiting on the spot). The door pin is read every loop (it's just a digital read, effectively instant).
2. **Validate** — each reading is checked against the last known-good timestamp. If a sensor hasn't produced a valid reading in 10 seconds (`SENSOR_STALE_MS`), it's flagged invalid rather than silently reusing an old number — this is what drives the "sensor warning" state on the dashboard and OLED.
3. **Detect events** — door open/close transitions, warning (30s) and alert (60s) door-open durations, and the start/end of a cooling-recovery episode are all detected here.
4. **Analyse** — the four score functions run: temperature score, door score, recovery score, and the energy model (which also produces the health score). Predictive maintenance is evaluated from the rolling 3-day recovery history.
5. **Persist** — small values (day counter, 3-day recovery history) are written to `Preferences` (the ESP32's flash-backed key-value store, NVS). Once a day rolls over, a summary record is appended to `/history.json` on LittleFS (a small flash filesystem), capped at 14 records so it never grows unbounded.
6. **Serve UI** — the `WebServer` instance answers HTTP requests on the SoftAP. The browser polls small JSON endpoints (`/data`, `/trend`, `/events`, `/dailyhistory`) every few seconds and redraws the page in place — no full page reload, no server push needed.

A `bool doorWasOpen` captured at the *top* of the loop (before any sensor read) is passed explicitly into the door and recovery functions, so door-close and recovery-start logic always sees a consistent before/after door state in the same pass — this fixed an earlier bug where the two functions disagreed about whether the door had just closed.

Feature 1: Temperature & Humidity Sensing

The DS18B20 is the source of truth for refrigerator temperature; the DHT22 supplies humidity and a secondary ambient-temperature reading. Both are sampled every 2 seconds.

Why non-blocking reads matter: the DS18B20's `requestTemperatures()` call takes up to ~750ms to convert a reading if you wait for it synchronously — long enough to stall the web server and make the dashboard feel frozen. Instead, the firmware calls `setWaitForConversion(false)`: it *requests* a conversion, moves on, and reads the result on the following cycle. The ESP32 is never stuck waiting.

Why "valid" is tracked separately from the reading itself: a sensor can physically disconnect (loose wire, probe failure) without the firmware noticing immediately — the last good value would otherwise just sit there looking current. Two flags, `temperatureReadingValid` and `humidityReadingValid`, are set true only when a fresh reading arrives, and are forced false if `SENSOR_STALE_MS` (10 seconds) passes with no update. The dashboard and OLED both check these flags and show a visible "Sensor error" / warning state instead of a frozen last-good number — so a disconnected sensor is obvious, not silently wrong.

Feature 2: Door Intelligence

The MC-38 reed switch gives a simple open/closed signal, but the firmware builds several layers of meaning on top of it:

- **Event counting.** Every open-to-closed-to-open transition increments `dailyDoorOpenings`. This resets at the start of each new day.
- **Duration tracking.** While the door is open, a per-opening timer (`currentDoorOpenSeconds`) counts up every second. When it closes, that duration is added to the day's running total and compared against the day's longest opening so far.
- **Two thresholds, two purposes:**
  - **30 seconds (`DOOR_WARNING_SECONDS`)** — crossing this turns the buzzer on (an audible nudge to shut the door) and marks the opening as a "long opening" for the day's tally.
  - **60 seconds (`DOOR_ALERT_SECONDS`)** — crossing this escalates the dashboard's door status from WARNING to ALERT and triggers the slide-down alert banner.
- **Hourly usage tracking.** Each door-open event is tallied against the current simulated/real hour, so the system can identify the single busiest hour of the day (`peakUsageHour`) — useful for spotting, e.g., "most openings happen around dinner time."

These per-day tallies (`dailyDoorOpenings` , `dailyLongDoorOpenings` , total open duration) feed directly into both the door component of the health score and the energy model below.

Feature 3: Cooling Recovery Analysis

What "recovery" means here: every time the door closes after being open and the temperature has drifted above `RECOVERY_TEMPERATURE` (4.5°C), the firmware starts a recovery timer. The timer stops the moment the temperature falls back to 4.5°C or below. That elapsed time is the *recovery time* — essentially, how long the compressor takes to undo the damage of a door-open event.

Why it's a useful signal on its own: recovery time is sensitive to things a single temperature reading can't show — a weakening compressor, a failing door seal, or overloading the fridge all tend to show up first as *slower recovery*, before the average temperature itself looks obviously wrong.

How each recovery is graded:

```
recovery time ⩽ 120s (GOOD_RECOVERY_SECONDS)      → status "GOOD"
120s < recovery time ⩽ 240s (MODERATE_RECOVERY_SECONDS) → status "MODERATE"
recovery time > 240s                               → status "SLOW"
```

Every completed recovery adds to `dailyRecoveryCount` and `dailyRecoveryTotal` (seconds), and the day's fastest/slowest recovery are tracked too. The **daily average** recovery time (`dailyRecoveryTotal ÷ dailyRecoveryCount`) is what actually feeds the recovery component of the health score, and — rolled up across three days — feeds predictive maintenance (next section).

Feature 4: Energy Waste Estimation

This is a model-based estimate, not a measurement. There's no current sensor (no ACS712) in this build — the kWh figure is calculated from how hard the day's conditions likely made the compressor work, using a reference wattage and an assumed duty cycle. It should always be presented as an estimate, never as a measured reading.

Step 1 — three "impact" scores (0–100, higher = worse):

```
temperatureEnergyImpact:
  temp ⩽ 5.0C → 0      5.0 < temp ⩽ 6.0C → 15
  6.0 < temp ⩽ 7.0C → 30     temp > 7.0C → 50

doorEnergyImpact = percentage of today's door-openings that were "long"
(⩾30s), capped at 50

recoveryEnergyImpact:
  avg recovery ⩽ 120s → 0     120 < avg ⩽ 240s → 15
  240 < avg ⩽ 360s → 30     avg > 360s → 50
```

Step 2 — combine into an energy waste score, then flip it into an efficiency score:

```
EnergyWasteScore = 0.40 \times TempImpact + 0.30 \times DoorImpact + 0.30
\times RecoveryImpact
EnergyEfficiencyScore = 100 - EnergyWasteScore
```

Step 3 — estimate actual kWh for the day, by converting waste into a duty-cycle adjustment on top of a baseline:

```
DutyCycle = 0.45 + (\text{+0.10 if temp} > 5C) + (\text{+0.10 if temp} > 6C)
+ \frac{DoorImpact}{1000} + \frac{RecoveryImpact}{1000}
EstimatedkWh = \frac{120W \times (HoursElapsedToday \times DutyCycle)}{1000}
```

Worked example (the same day used throughout this document): temperature reading 4.2°C, 10 door openings with 2 long ones (20%), average recovery 150s, 24 hours elapsed.

- Impacts: temperature = 0 (4.2 ⩽ 5.0), door = 20, recovery = 15 (150s falls in the 120–240s band)
- `EnergyWasteScore = 0.40(0) + 0.30(20) + 0.30(15) = 10.5` → `EnergyEfficiencyScore = 89.5`
- `DutyCycle = 0.45 + 0 + 0 + 0.020 + 0.015 = 0.485`
- `EstimatedkWh = 120 × (24 × 0.485) / 1000 ≈ 1.40 kWh for the day`

The energy *status* label shown on the dashboard (LOW / MODERATE / HIGH / VERY HIGH) is just the `EnergyWasteScore` bucketed at 20/40/60 — here, 10.5 → LOW.

Feature 5: Predictive Maintenance

The idea: if the average daily recovery time is *consistently rising* over several days, that's an early sign of a weakening compressor or a failing door seal — worth flagging before it becomes an obvious fault.

Why only days with a recovery event count. A day with zero door-triggered recoveries isn't a "perfect" day for this purpose — it's a day with no data. Counting it as 0 seconds would fake an improving trend. So `validRecoveryDays` only increments on a day that actually had at least one recovery event, and only those days' averages are stored into the rolling 3-slot history (`recoveryDay1` , `recoveryDay2` , `recoveryDay3`).

The trigger logic:

```
IF validRecoveryDays < 3:
    status = "INSUFFICIENT HISTORY"
ELSE IF recoveryDay1 < recoveryDay2 < recoveryDay3  (strictly increasing)
   AND (recoveryDay3 - recoveryDay1) / recoveryDay1 ⩾ 15%:
    status = "MAINTENANCE ADVISED"
ELSE:
    status = "NORMAL"
```

Worked example: three valid days with average recovery times of 100s, 120s, then 140s.

- Strictly increasing? 100 < 120 < 140 → yes
- Percentage increase: `(140 − 100) / 100 = 40%` , which is ⩾ 15%
- Result: **MAINTENANCE ADVISED** — the dashboard's alert banner slides down and an activity-feed entry is logged the moment this first triggers.

If instead the three days were 100s, 95s, 110s — not strictly increasing — the result stays NORMAL even though the last value is higher than the first, because a single noisy day shouldn't trigger a maintenance flag on its own.

Feature 6: Health Score Calculation

The health score is a single 0–100 number that rolls up all four areas the system watches, weighted by how much each matters:

```
HealthScore = 0.40 \times TempScore + 0.20 \times DoorScore + 0.20 \times
RecoveryScore + 0.20 \times EnergyScore
```

Each sub-score, in plain terms:

Component	Weight	How it's scored
Temperature stability	40%	Based on the current reading: 100 if 3–5°C, 85 if ≤6°C, 65 if ≤7°C, 40 if ≤8°C, 20 above that.
Door behaviour	20%	Based on what % of today's door-openings were "long" (≥30s): 100 if ≤10%, 85 if ≤25%, 65 if ≤50%, else 40.
Cooling recovery	20%	Based on today's average recovery time: 100 if ≤120s, 75 if ≤240s, 50 if ≤360s, else 25.
Energy efficiency	20%	`100 - EnergyWasteScore` from the energy model (Feature 4).

All four use **today's** figures, not lifetime totals — this was a deliberate fix: an earlier draft mixed daily and all-time-cumulative stats, which made the score mean something different depending on how long the device had been running. Keeping everything daily means the score always answers the same question: *"how did the fridge do today?"*

Full worked example (continuing the same day used in Features 3–5): temperature 4.2°C, 10 door openings with 2 long (20% → door score 85), average recovery 150s (≤240s → recovery score 75), energy efficiency 89.5 (from Feature 4).

```
TempScore     = 100   (4.2°C is in the 3–5°C band)
DoorScore     = 85    (20% long openings is in the ≤25% band)
RecoveryScore = 75    (150s average is in the ≤240s band)
EnergyScore   = 89.5

HealthScore = 0.40(100) + 0.20(85) + 0.20(75) + 0.20(89.5)
            = 40 + 17 + 15 + 17.9
            = 89.9  →  rounds to 90  →  status "EXCELLENT" (≥85)
```

Status bands: ≥85 EXCELLENT · 70–84 GOOD · 50–69 MODERATE · below 50 POOR.

Feature 7: Persistence & Day Tracking

Why there's no RTC and no NTP. A real-time clock chip adds cost and wiring; NTP (network time sync) needs an internet connection — but this device runs as its own Wi-Fi hotspot (SoftAP) with no uplink to the internet at all. Both options were ruled out for the same underlying reason.

The software day counter, instead. The firmware tracks a `dayNumber` and `dailyElapsedMs` purely from `millis()` (the ESP32's running clock since boot). Once `dailyElapsedMs` reaches 24 hours' worth of milliseconds, the day rolls over: daily counters reset, a summary is appended to history, and `dayNumber` increments. This doesn't know the real calendar date, but it doesn't need to — it only needs to know *that a day has passed*, consistently.

Surviving a restart. `dayNumber`, `dailyElapsedMs`, and the 3-slot predictive-maintenance recovery history (`recoveryDay1/2/3`, `validRecoveryDays`) are all written to `Preferences` — the ESP32's flash-backed key-value store (built on NVS, Non-Volatile Storage). So a power cycle or reset doesn't lose the day count or the maintenance-trend history, only the in-RAM live buffers (trend sparkline, activity feed) — which is acceptable since those are meant to show *recent* activity, not a permanent record.

Daily history, separately. Each day's final summary (health score, min/max temperature, estimated kWh, etc.) is appended as one JSON record to `/history.json` on LittleFS, a small flash filesystem. The file is capped at 14 records — once full, the oldest record is dropped before the newest is appended — so it never grows without bound. This file is what powers the dashboard's 7-day health-score bar chart.

Feature 8: The Local Dashboard

SoftAP vs. normal Wi-Fi. Most ESP32 projects use *Station mode* — the device joins an existing Wi-Fi network, like a phone does. This project instead uses **SoftAP (Access Point) mode**: the ESP32 *becomes* its own Wi-Fi network (`Smart_Refrigerator`). A phone or laptop connects to it directly, then browses to `http://192.168.4.1`. This was chosen deliberately for demo/viva reliability — it works identically in any room, with no dependency on venue Wi-Fi credentials, captive portals, or an internet connection existing at all.

Why the dashboard has zero external dependencies. Because SoftAP clients have no internet uplink, the page can't load Chart.js, Google Fonts, or any CDN-hosted library the way a typical "modern" dashboard would — those requests would simply fail. Every visual element (the animated health-score ring, the temperature sparkline, the 7-day bar chart) is hand-built in plain HTML/CSS/JavaScript/SVG/Canvas, inlined directly into the page the ESP32 serves. This guarantees the dashboard always loads instantly over the local network, with nothing to fetch from anywhere else.

How the live data gets to the browser. The page doesn't reload — it polls small JSON endpoints in the background and redraws just the parts that changed:

Endpoint	Polled every	What it returns
`/data`	3s	Current readings, all scores, statuses, sensor validity flags
`/trend`	4s	The in-RAM temperature ring buffer (~90 samples, ~3 minutes) — feeds the sparkline
`/events`	6s	The last 8 activity-log entries (door/recovery/maintenance events), newest first
`/dailyhistory`	20s	The LittleFS `history.json` records — feeds the 7-day health-score bar chart

On-device fallback. Everything shown on the dashboard is also summarised on the OLED (temperature, humidity, door state, health score, estimated kWh, day number) and the buzzer sounds on a long door-open — so the key status is visible even with no phone connected at all.

Viva Model Questions & Answers

Q1. Why did you choose an ESP32 over an Arduino Uno? The ESP32 has built-in Wi-Fi (needed for the dashboard), far more RAM and flash than an Uno, and native I²C/1-Wire support for the OLED and DS18B20 — all without extra shield hardware. An Uno would need a separate Wi-Fi module and still be tight on memory for a web server plus history logging.

Q2. Why Access Point mode instead of connecting to an existing Wi-Fi network? Station mode depends on a specific network being available with known credentials — risky in a viva room or anywhere the venue Wi-Fi is locked down. SoftAP makes the ESP32 its own self-contained network, so the demo works identically anywhere, with a phone connecting directly to it.

Q3. Your energy figure — is it measured or calculated? Calculated. There's no current/power sensor (e.g. ACS712) in this build. It's a duty-cycle model: a reference compressor wattage (120W) multiplied by an estimated "on" fraction of the day, where that fraction rises with worse temperature, more long door-openings, and slower recovery. It's explicitly labelled an estimate on the dashboard, never shown as a direct measurement.

Q4. Why weight temperature at 40% and the other three factors at 20% each in the health score? Temperature stability is the refrigerator's core job — everything else (door behaviour, recovery speed, energy use) is really a *symptom* of how well temperature is being held. Giving it the largest single weight reflects that it's the most direct indicator of appliance health, while still letting the other three meaningfully move the score.

Q5. How do you detect a disconnected or faulty sensor? Each sensor reading is timestamped. If no valid reading arrives for 10 seconds (`SENSOR_STALE_MS`), a `...Valid` flag flips false, and both the OLED and the dashboard switch to a visible "sensor error/warning" state instead of continuing to display the last good value as if it were current.

Q6. What's the difference between the door being "open" and the door causing an "alert"? Any open door is just OPEN. If it stays open ⩾ 30 seconds, the buzzer sounds and the dashboard shows WARNING. If it stays open ⩾ 60 seconds, the status escalates to ALERT and the dashboard's slide-down alert banner appears — two graduated thresholds, not one.

Q7. How does predictive maintenance avoid false alarms from a single bad day? It requires three consecutive valid days (days that actually had at least one recovery event) with *strictly increasing* average recovery time, and the total increase from day 1 to day 3 must be ⩾15%. A single noisy or non-monotonic day can't trigger it on its own.

Q8. Why is there no RTC chip for real dates? An RTC adds cost and wiring, and NTP (the usual software alternative) needs an internet connection — which SoftAP mode doesn't have. Instead, the firmware tracks elapsed time since boot (`millis()`) and treats every 24 hours of elapsed time as one "day," persisting that counter to flash so it survives a restart. It doesn't know the calendar date, but it doesn't need to for day-over-day comparisons.

Q9. Why does the DS18B20 go inside the fridge but the DHT22 doesn't need to? The DS18B20 probe is waterproof and is the system's primary, authoritative temperature reading, so it's placed directly in the cold compartment. The DHT22 (not waterproof) stays outside, mounted near the door seal, where it still usefully reads ambient humidity and temperature without condensation risk.

Q10. What happens to your data if the ESP32 loses power? The day counter, elapsed-time-in-day, and the 3-day predictive-maintenance history are written to `Preferences` (flash-backed NVS), so they survive a reset. Completed days are separately appended to

`/history.json` on LittleFS. Only short-lived in-RAM buffers — the live trend sparkline and the activity feed — are lost on restart, which is acceptable since they're meant to show recent activity, not permanent history.

Q11. Why cap the history file at 14 records instead of keeping everything forever? LittleFS is a small flash filesystem with limited space on an ESP32. An unbounded log would eventually fail to write. Fourteen days is enough for the 7-day dashboard chart (with headroom) while keeping the file small and the write/trim operation fast.

Q12. What did you deliberately leave out of scope, and why? Food-freshness prediction, cloud connectivity, direct compressor/mains control, and measured (vs. estimated) power consumption. Each would have added hardware, cost, or risk (mains switching especially) disproportionate to what the team could reliably implement and validate within the project's scope and timeline.

Q13. How is this different from just a thermometer with Wi-Fi? A thermometer reports one number. This system correlates door behaviour, recovery speed, and temperature into a single interpretable score, estimates energy impact from those same behaviours, and flags a trending compressor problem before it becomes an obvious failure — all computed locally, with no cloud dependency.

Q14. Is the health score a certified diagnostic value? No — it's explicitly described as a transparent, weighted heuristic indicator for this project, not a certified appliance diagnostic. Its purpose is to be interpretable (every input is explainable) rather than clinically validated.